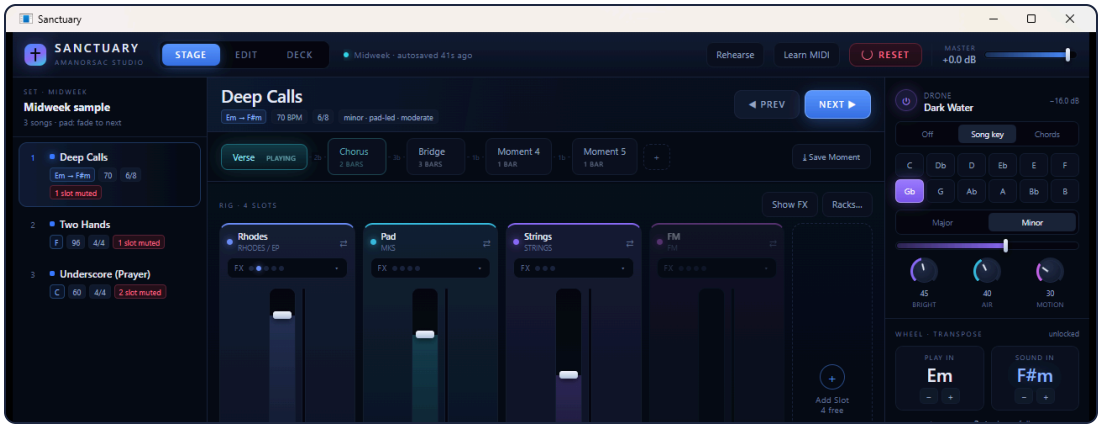
SANCTUARY

Live-service keys instrument (Amanorsac Studio) — internal product-analysis brief for Amanorsac Studio company-wide standards

IN DEVELOPMENT



Main interface — Stage screen, captured live from the built v1.0.0 standalone window, 2026-09-03.

1 · PRODUCT IDENTITY

Product name	Sanctuary
Category	VST3 / Standalone plugin (AU spec'd for macOS, not built in this tree) — live-performance keys/pad instrument.
One-sentence	A live-worship/stage keyboard instrument: Sets of Songs made of glide-able Moments, a chord-following drone and performance pads, run from one screen during a live set.
Primary purpose	Real-time performance playback — the "player" side of Amanorsac's pipeline, reading libraries the Amanorsac Studio sampler app produces.
Intended users	Worship/stage keyboardists running a whole set without looking away from one screen — performer-facing, unlike the internal-only sampler app.
Status	In Development — functional VST3 + Standalone build (v1.0.0), 58 passing automated unit tests. No licensing, installer signing, or auto-update; still reads plain `.jmi` folders (the encrypted `.aofas` reader exists as a separate package but is not wired in yet).

2 · WHAT THE PRODUCT DOES

Main function	Load a Set of Songs, play each Song's Moments live with glides, chord-following drone and pads, move Prev/Next through a set with no audio cutoff.
Key features	Stage screen (rig of up to 8 slots, drone, wheel/transpose, tempo, live trim, performance pads) · Edit screen · Deck (song/set rail) · Surfaces (macro bindings) · MIDI Learn · Rehearse mode (stops session writes) · session autosave + "resume where I was" on relaunch · RESET (panic + drone fade).
Typical workflow	Pick a Set → a Song loads its rig/drone → Moments glide the sound through the song's sections → Prev/Next moves between Verse/Chorus/Bridge → pads punctuate transitions → Edit screen used between services to build content.
Distinctive	No-cutoff double-buffered slot engine chains, so a Song swap mid-performance never clicks or drops a held note. Drone harmonizes live to the chord being played. UI is the literal HTML/CSS/JS mockup running inside a JUCE WebView (not native-redrawn), bridged through one `cmd()` call and a parameter-path space shared with Surfaces/macros/Moments.

4 · VISUAL / UX IDENTITY (AS CURRENTLY IMPLEMENTED)

bg · panel · accent (blue) · ice (info) · violet (rehearse) · ember (learn/reset) · slot accent s2 · slot accent s4

Background	Near-black navy gradient chassis, thin hairline borders, flatter/denser than the sampler app's photoreal hardware register.
Typography	Inter throughout (13px base) — one utilitarian body face, no display/condensed headline face.
Controls	Vertical slot faders in the rig mixer; rotary knobs for drone Bright/Air/Motion and Live Trim; horizontal faders elsewhere; gradient-filled segmented page tabs (Stage/Edit/Deck); bordered "tool" chips (Rehearse, Learn MIDI) with an on-state glow.
Nav	Top segmented Stage/Edit/Deck tabs, tool chips and RESET beside them, master fader top right.
Meters	Master level horizontal meter (header); per-slot dB readouts under each fader.
States	Learn-mode chip blinks (1s keyframe) while armed; on-state chips get a colour-matched glow border; six distinct slot-accent hues (s1–s6) identify worlds/slots at a glance.
Motion	9 CSS transitions, 2 keyframe animations total (blink on Learn) — restrained, control-room feel over ornamentation.

5 · TECHNICAL SNAPSHOT

Framework	JUCE 8 (C++17); UI is HTML/CSS/JS hosted in `juce::WebBrowserComponent` (WebView2 on Windows, WKWebView on macOS).
Platforms	Windows build verified in this tree (VST3 + Standalone artifacts present); macOS path is coded but not confirmed built here.
Plugin formats	VST3, Standalone confirmed built; AU specified in CMake but requires macOS to produce.
Major deps	JUCE 8 (own checkout, shared read-only with JM Signature); own copy of the JM Signature DSP engine headers (own fork, JM Signature untouched); WebView2 SDK (Windows).
Audio/DSP	Double-buffered slot chains (SamplerEngine + FxRackDsp per slot) for glitch-free Song switches; drone synth (two crossfaded ground+air beds); chord-follow template matcher; 30 Hz message-thread Moment glides; host-automatable master/limiter/4 macros, Rig values automatable via macros/Surfaces.
Auth/licensing	A plain `licenseKey` string field exists in storage — its own code comment states "no cryptography, no server activation, no obfuscation, no enforcement." A placeholder, not a working system.
Installer/update	None observed in this app (no NSIS script here); unsigned; no auto-updater.
External deps	None at runtime — local `.jmi` library + session/settings JSON on disk. The sealed `.aofas` reader (packages/aofas-format) is built and tested but not yet wired into this app's loading path.

KEEP

- Chord-follow drone, no-cutoff double-buffered slot chains, Moments/Surfaces model — genuinely Sanctuary-specific
- WebView-hosted UI + single `cmd()` bridge over a shared parameter-path space

STANDARDIZE

- Blue beam palette (already shared with the sampler app)
- Segmented-tabs + tool-chip + status-line chassis pattern (reconcile with the sampler app's header)
- Session-file "resume where I was" reliability pattern

MISSING

- Account integration; real licensing (field is an unenforced placeholder)
- `.aofas` support — reader exists, not wired in yet
- Installer signing, auto-updater
- About screen, legal links, a11y pass, crash/telemetry

Source: apps/sanctuary (repo). Generated from current code & a live build-window screenshot — no invented or planned-only features included.